TRAINING CENTER MANAGEMENT SYSTEM (TCMS) MISSION SPECIFICATIONS

Scenario
You are a software developer at Company X, a technology company specializing in educational and training solutions. Your mission is to build a Training Center Management System Web API using ASP.NET Core Web API and Entity Framework Core.

Technology Stack

- ASP.NET Core Web API
- Entity Framework Core
- LINQ
- Fluent API
- Data Annotations
- DTOs
- AutoMapper
- Dependency Injection
- Swagger

System Models

CourseCategory

Property	Data Type	Validation Rules
Id	int	Primary key, auto-generated
Name	string	Required, maximum length 100, cannot be empty
Description	string?	Optional (nullable), maximum length 300

Instructor

Property	Data Type	Validation Rules
Id	int	Primary key, auto-generated
FullName	string	Required, maximum length 150, cannot be empty
Email	string	Required, valid email address
Specialization	string	Required, maximum length 100
HireDate	DateTime	Required

Student

Property	Data Type	Validation Rules
Id	int	Primary key, auto-generated
FullName	string	Required, maximum length 150, cannot be empty
Email	string	Required, valid email address
PhoneNumber	string?	Optional (nullable), maximum length 20
RegistrationDate	DateTime	Required

Course

Property	Data Type	Validation Rules
Id	int	Primary key, auto-generated
Title	string	Required, maximum length 200, cannot be empty
DurationHours	int	Required, value between 1 and 200
Fee	decimal	Required, must be greater than 0
Capacity	int	Required, value between 1 and 100
IsActive	bool	Required
CategoryId	int	Foreign key to CourseCategory
InstructorId	int	Foreign key to Instructor

Enrollment

Property	Data Type	Validation Rules
Id	int	Primary key, auto-generated
EnrollmentDate	DateTime	Required
Status	EnrollmentStatus	Required: Active, Completed, or Cancelled
Grade	decimal?	Optional (nullable), value between 0 and 100
StudentId	int	Foreign key to Student
CourseId	int	Foreign key to Course

Relationships

- **CourseCategory -> Courses** (One-to-Many)
- **Instructor -> Courses** (One-to-Many)
- **Student -> Enrollments** (One-to-Many)
- **Course -> Enrollments** (One-to-Many)
- **Student <-> Course** (Many-to-Many using Enrollment)

Navigation Properties

Add the appropriate navigation properties so that each relationship can be traversed in both directions. Your models should support the following navigation structure:

- CourseCategory -> collection of Courses
- Instructor -> collection of Courses
- Student -> collection of Enrollments
- Course -> Category, Instructor, and collection of Enrollments
- Enrollment -> Student and Course

DTOs

Do not return Entity Models directly from controllers when a DTO is required. Create and use the following DTOs.

CourseCategory DTOs

DTO	Properties
CourseCategoryDTO	Id, Name, Description
CreateCourseCategoryDTO	Name, Description
UpdateCourseCategoryDTO	Name, Description

Instructor DTOs

DTO	Properties
InstructorDTO	Id, FullName, Email, Specialization, HireDate, CourseCount
CreateInstructorDTO	FullName, Email, Specialization, HireDate
UpdateInstructorDTO	FullName, Email, Specialization, HireDate

Student DTOs

DTO	Properties
StudentDTO	Id, FullName, Email, PhoneNumber, RegistrationDate, EnrolledCoursesCount
CreateStudentDTO	FullName, Email, PhoneNumber
UpdateStudentDTO	FullName, Email, PhoneNumber

Course DTOs

DTO	Properties
CourseDTO	Id, Title, DurationHours, Fee, Capacity, IsActive, CategoryName, InstructorName, EnrolledStudentsCount
CreateCourseDTO	Title, DurationHours, Fee, Capacity, CategoryId, InstructorId
UpdateCourseDTO	Title, DurationHours, Fee, Capacity, IsActive, CategoryId, InstructorId

Enrollment DTOs

DTO	Properties
EnrollmentDTO	Id, StudentName, CourseTitle, EnrollmentDate, Status, Grade
CreateEnrollmentDTO	StudentId, CourseId
CompleteEnrollmentDTO	Grade

AutoMapper Requirements

Configure mappings for the DTOs. Mappings should include the following directions and any custom member mappings required for calculated or related values.

- CourseCategory -> CourseCategoryDTO; CreateCourseCategoryDTO -> CourseCategory; UpdateCourseCategoryDTO -> CourseCategory
- Instructor -> InstructorDTO; CreateInstructorDTO -> Instructor; UpdateInstructorDTO -> Instructor
- Student -> StudentDTO; CreateStudentDTO -> Student; UpdateStudentDTO -> Student
- Course -> CourseDTO; CreateCourseDTO -> Course; UpdateCourseDTO -> Course
- Enrollment -> EnrollmentDTO; CreateEnrollmentDTO -> Enrollment
- Map related values such as CategoryName and InstructorName, and calculated counts such as CourseCount or EnrolledStudentsCount.

Fluent API

Configure the following additional constraints using Fluent API:

- **CourseCategory.Name** must be unique.
- **Student.Email** must be unique.
- **Instructor.Email** must be unique.
- A student cannot enroll in the same course twice. Configure a unique composite index on **StudentId** + **CourseId**.
- Configure **Course.Fee** with precision **10,2**.
- Set the default value of **Student.RegistrationDate** to the current date and time.
- Set the default value of **Enrollment.EnrollmentDate** to the current date and time.
- Set the default value of **Course.IsActive** to **true**.
- Configure the CourseCategory-Course, Instructor-Course, Student-Enrollment, and Course-Enrollment relationships.
- Deleting an Instructor or CourseCategory must not automatically delete its Courses. Configure an appropriate DeleteBehavior.

Controllers

- CourseCategoriesController
- CoursesController
- InstructorsController
- StudentsController
- EnrollmentsController

Endpoints

#	Requirement	Endpoint
1	Get all courses with a fee greater than the specified value. Return CourseDTO results.	`GET /api/courses/fee-greater-than/{fee}`
2	Get all courses whose fee is between a minimum and maximum value.	`GET /api/courses/fee-between?min=300&max;=1000`
3	Search for courses whose title contains a specific keyword. Make the search case-insensitive if possible.	`GET /api/courses/search?title=asp`
4	Get the first course in the collection. Return 404 Not Found if there are no courses.	`GET /api/courses/first`
5	Get the first active course where IsActive is true.	`GET /api/courses/first-active`
6	Get a course by ID as CourseDTO, including CategoryName, InstructorName, and EnrolledStudentsCount. Return 404 if not found.	`GET /api/courses/{id}`
7	Get the last course in the collection.	`GET /api/courses/last`
8	Check whether a course with the given ID exists. Return true or false.	`GET /api/courses/exists/{id}`
9	Verify that all courses are active. Return true or false.	`GET /api/courses/all-active`
10	Retrieve only the titles of all courses.	`GET /api/courses/titles`
11	Get all courses sorted by title in ascending order.	`GET /api/courses/sort/title`
12	Find the lowest course fee.	`GET /api/courses/lowest-fee`
13	Find the average course fee.	`GET /api/courses/average-fee`
14	Retrieve courses using pagination. Page 1 returns the first page; page 2 returns the next page. Use Skip and Take in the correct order.	`GET /api/courses/paged?pageNumber=1&pageSize;=5`
15	Get the number of courses in each category. Return the category name and course count.	`GET /api/coursecategories/course-count`

#	Requirement	Endpoint
16	Get each instructor with the number of courses they teach.	`GET /api/instructors/course-count`
17	Create a new Course Category using CreateCourseCategoryDTO. Category name must be unique. Return 400 for duplicate names and 201 Created on success.	`POST /api/coursecategories`
18	Create a Student using CreateStudentDTO. Email must be unique. RegistrationDate should be generated automatically. Return 400 for duplicate email and 201 Created on success.	`POST /api/students`
19	Create a Course using CreateCourseDTO. Validate that Category and Instructor exist, Fee > 0, and Capacity > 0. Return 404 when a required related entity does not exist and 201 Created on success.	`POST /api/courses`
20	Update a Course using UpdateCourseDTO. Validate that the Course, Category, and Instructor exist. Return 404 when required data does not exist and 204 No Content on success.	`PUT /api/courses/{id}`
21	Enroll a Student in a Course using CreateEnrollmentDTO. Validate Student exists, Course exists, Course is active, Student is not already enrolled, and capacity is available. On success set Status = Active and use the current EnrollmentDate. Return 404 for missing Student/Course, 400 for business-rule failures, and 201 Created on success.	`POST /api/enrollments`
22	Complete an Enrollment using CompleteEnrollmentDTO. Enrollment must exist and have Active status. Grade must be between 0 and 100. Set Status = Completed and save the grade. Return 404 if not found, 400 if already Completed/Cancelled, and 200 OK on success.	`PUT /api/enrollments/{id}/complete`
23	Cancel an Enrollment. Enrollment must exist and currently be Active. Set Status = Cancelled. Return 404 if not found, 400 if already Completed/Cancelled, and 200 OK on success.	`PUT /api/enrollments/{id}/cancel`
24	Delete an Enrollment only when Status is Cancelled. Return 404 if not found, 400 for Active or Completed records, and 204 No Content when deleted.	`DELETE /api/enrollments/{id}`
25	Get all courses registered by a specific Student. Return CourseTitle, InstructorName, EnrollmentDate, Status, and Grade through DTOs. Return 404 if the Student does not exist.	`GET /api/students/{id}/courses`

Data Seeding

Seed enough initial data to test filtering, sorting, aggregates, pagination, relationships, and enrollment operations. Seed at least the following entity types:

- CourseCategories
- Instructors
- Students
- Courses

Additional Requirements

- Configure DbContext using Dependency Injection.
- Configure all required relationships and Fluent API rules.
- Apply Data Annotations for validation.
- Create and use DTOs.
- Configure AutoMapper.
- Do not return Entity Models directly when a DTO is required.
- Seed initial data.
- Create EF Core Migrations and update the database.
- Validate inputs and return proper HTTP status codes.
- Use asynchronous EF Core methods whenever appropriate.

- Test all endpoints using Swagger or Postman.
- Manage the project using Git.

Deliverables

- Entity Models
- DTOs
- AutoMapper Profile
- DbContext
- Fluent API Configuration
- Data Seeding
- EF Core Migrations
- Controllers
- LINQ Endpoints
- Swagger Testing